

HW 3: CUDA Core 및 Tensor Core를 사용한 Jacobi Iteration의 구현

과목 : 기초 GPU 프로그래밍

교수 : 임인성 교수님

학과 : 컴퓨터공학과

학번 : 20211562

이름 : 이동협

실험 환경

GPU : Geforce RTX 4070 Ti

CUDA Compute Capability : 8.9

측정 방법

CUDA Core 방법에서는 16×16 tile을 shared memory에 올려 matrix 곱셈을 수행했다. 각 Jacobi 반복에서는 $D = A * B + C$ 를 계산한 뒤 $B = A * D + C$ 를 계산하여 두 번의 iteration을 한 loop에서 처리했다.

Tensor Core 방법에서는 wmma를 사용했다. A와 B는 TF32 precision으로 변환된 input fragment로 사용하고 accumulator fragment는 float으로 선언하여 FP32 누적 계산을 수행했다. 실험환경의 CC가 8.9로 8.0이상이었기에 과제에서 주어진 FP16 대신 TF32 사용 권장을 지켰다.

Residual norm 계산은 $R = U * B - V$ 를 계산한 뒤 infinity norm을 구했다. residual 계산은 아주 작은 차이를 다루기에 CUDA Core 경로에서 double accumulation을 사용하여 CPU 기준 residual과 일치하도록 했다.

구현 방법

Jacobi.cpp

void Jacobi::Jacobi_iteration_CC(Status& flag)

이 함수는 CUDA Core Jacobi iteration의 진입점이다.

먼저 $V_norm = ||V||$ 를 계산하여 relative residual norm 계산에 사용하고 이후에 Jacobi_iteration_CC_cuda() wrapper를 호출하여 실제 GPU 연산을 수행한다.

GPU 실행이 끝나면 status_code를 Jacobi::Status 값으로 변환하여 flag에 저장한다.

void Jacobi::Jacobi_iteration_TC(Status& flag)

이 함수는 Tensor Core Jacobi iteration의 진입점이다.

구조는 Jacobi_iteration_CC()와 동일하지만 내부적으로 Jacobi_iteration_TC_cuda()를 호출한다. 이 wrapper는 CUDA Core tiled GEMM 대신 WMMA 기반 Tensor Core TF32 GEMM을 사용한다.

A와 B는 Tensor Core WMMA kernel 내부에서 TF32 input fragment로 사용되고 accumulator 및 출력은 FP32로 유지된다. RTX 4070 Ti의 Compute Capability가 8.9이므로 FP16 대신 TF32를 사용했다.

my_kernels.cu

my_kernels.cu는 CUDA로 컴파일되어야 하는 GPU kernel과 wrapper 함수들을 모아 둔 파일이다.

Jacobi.cpp는 일반 C++ 파일이므로 `__global__` kernel이나 WMMA 코드를 직접 작성하기 어렵다. 따라서 Jacobi.cpp의 두 함수는 진입점 역할만 하고 실제 GPU 연산은 `my_kernels.cu`에서 수행한다.

`__global__ void matmul_cuda_kernel(float* Out, const float* A, const float* B, const float* C, int n, int op)`

CUDA Core 기반 tiled matrix multiplication kernel이다. TILE = 16으로 설정되어 있으며 각 thread block은 16 x 16 출력 tile 하나를 계산한다. A와 B의 일부 tile을 shared memory에 올린 뒤 inner product를 수행한다.

op 값에 따라 두 가지 연산을 수행한다.

OP_FMA -> $Out = A * B + C$

OP_RESIDUAL -> $Out = A * B - C$

Jacobi update에서는 OP_FMA를 사용하여 $D = A*B + C$, $B = A*D + C$ 를 계산한다. Residual 계산에서는 OP_RESIDUAL을 사용하여 $R = U*B - V$ 를 계산한다. Residual은 매우 작은 값을 비교해야 하므로 residual 경로에서는 내부 누산을 double로 수행하여 CPU residual 계산과 일치하도록 했다.

`__global__ void row_abs_sum_and_max_kernel(const float* matrix, float* norm, int n)`

matrix의 infinity norm을 계산하기 위한 kernel이다. 하나의 block이 matrix의 한 row를 담당한다. 각 thread가 row의 여러 column을 나누어 읽고 절댓값을 합산한다. 이후 block 내부 reduction을 통해 한 row의 절댓값 합을 구한다.

마지막으로 각 row sum 중 최댓값을 `atomicMaxFloatNonnegative()`를 통해 norm에 저장한다. matrix 원소 절댓값 합은 항상 0 이상이므로 non-negative float 값에 대해 atomic max를 적용할 수 있다.

`__device__ void atomicMaxFloatNonnegative(float* address, float value)`

CUDA에서 float 값에 대한 atomic max를 수행하기 위한 device 함수이다. row sum은 항상 0 이상이므로 float bit pattern을 int로 해석하여 atomicCAS 기반으로 최대값을 갱신한다. 이 함수는 infinity norm 계산에서 여러 block이 동시에 global maximum을 갱신할 때 사용된다.

`__global__ void wmma_gemm_tf32_kernel(float* D, const float* A, const float* B, const float* C, int n)`

Tensor Core WMMA 기반 TF32 GEMM kernel이다. 각 warp가 16 x 16 출력 tile 하나를 계산한다. TF32 WMMA는 16 x 16 x 8 fragment를 사용하므로, k 방향으로 8개씩 이동하면서 multiply-accumulate를 반복한다.

사용한 fragment는 다음과 같다.

`wmma::fragment<wmma::matrix_a, 16, 16, 8, wmma::precision::tf32, wmma::row_major>`

`wmma::fragment<wmma::matrix_b, 16, 16, 8, wmma::precision::tf32, wmma::row_major>`

`wmma::fragment<wmma::accumulator, 16, 16, 8, float>`

input matrix A, B는 TF32 precision으로 사용하고 accumulator는 FP32이다. C를 accumulator에 먼저 load한 뒤 `mma_sync()`를 반복하므로 최종 결과는 $D = A * B + C$ 가 된다.

`float infinity_norm_on_device(float* d_norm, const float* d_matrix, int n)`

device에 있는 maxtrix의 infinity norm을 계산하는 helper 함수. 먼저 `d_norm`을 0으로 초기화하고 `row_abs_sum_and_max_kernel()`을 실행한다. kernel 실행 후 `d_norm` 하나만 host로 복사하여 최종 norm 값을 반환한다.

`void run_jacobi(bool use_tensor_cores, float* B, float* D, float* R, const float* A, const float* C, const float* U, const float* V, size_t matrix_size, int max_iterations, float epsilon, float v_norm, int* completed_iterations, float* final_relative_residual_norm, int* status_code)`

CUDA Core와 Tensor Core에서 공통으로 사용하는 Jacobi 반복 실행 함수이다. 먼저 필요한 maxtrix들을 device memory에 할당하고 host에서 device로 복사한다. 이후 다음 과정을 반복한다.

$R = U * B - V$

$relative_residual = ||R|| / ||V||$

if $relative_residual < epsilon$:

 success

if $iteration_count \geq max_iterations$:

 fail

$D = A * B + C$

$B = A * D + C$

$iteration_count += 2$

`use_tensor_cores == false`이면 `matmul_cuda_kernel()`을 사용하여 CUDA Core 연산을 수행한다.

use_tensor_cores == true이면 wmma_gemm_tf32_kernel()을 사용하여 Tensor Core 연산을 수행한다.

반복이 끝나면 최종 B, D, R을 host로 복사하고 수행 iteration 수, 최종 residual, 성공/실패 상태를 return한다.

Jacobi_iteration_CC_cuda

```
extern "C" void Jacobi_iteration_CC_cuda (float* B, float* D, float* R, const float* A,  
    const float* C, const float* U, const float* V, size_t matrix_size, int max_iterations,  
    float epsilon, float v_norm, int* completed_iterations, float* final_relative_residual_norm,  
    int* status_code)
```

Jacobi::Jacobi_iteration_CC()에서 호출되는 CUDA Core wrapper이다. 내부적으로 run_jacobi(false, ...)를 호출한다. 따라서 Jacobi update는 CUDA Core FP32 tiled GEMM으로 수행된다.

Jacobi_iteration_TC_cuda

```
extern "C" void Jacobi_iteration_TC_cuda (float* B, float* D, float* R, const float* A,  
    const float* C, const float* U, const float* V, size_t matrix_size, int max_iterations,  
    float epsilon, float v_norm, int* completed_iterations, float* final_relative_residual_norm,  
    int* status_code)
```

Jacobi::Jacobi_iteration_TC()에서 호출되는 Tensor Core wrapper이다. 내부적으로 run_jacobi(true, ...)를 호출한다. 따라서 Jacobi update는 Tensor Core TF32 WMMA kernel로 수행된다.

측정 결과 및 분석

Matrix Size	CPU time (ms)	CUDA Core time (ms)	TensorCore time (ms)	CPU residual	CUDA residual	Tensor residual
256	787.862	112.909	5.608	3.052558 e-08	3.649433 e-08	1.793526 e-05
512	6939.589	136.682	22.017	2.977081 e-08	3.490851 e-08	1.264876 e-05
1024	54555.988	189.479	375.566	4.960132 e-08	5.234583 e-08	1.008106 e-05
2048	521829.156	557.857	1239.549	3.037843 e-08	3.589873 e-08	1.214108 e-05

CUDA 버전은 모든 matrix 크기에서 CPU보다 훨씬 빠르게 수행되었으며 residual도 CPU와 같은 1e-8 수준으로 나타났다. 따라서 FP32 기반 CUDA Core 구현은 속도/정확도 모두에서 좋은 결과를 보였다.

Tensor 버전은 작은 matrix 크기에서는 가장 빠른 수행 시간을 보였다. 그러나 TF32 input precision을 사용하기 때문에 residual은 약 1e-5 수준에 머물렀고 epsilon = 1e-7 조건을 만족하지 못했다. 이는 accumulator가 FP32이더라도 입력 A와 B가 TF32 precision으로 사용되기 때문에 곱셈 단계에서 FP32 보다 낮은 precision을 갖기 때문이다.

matrix size가 커질수록 Tensor 시간이 CUDA보다 커지는 경우가 있었다. 이는 구현이 cuBLAS와 같은 최적화된 GEMM이 아니라 단순 WMMA kernel이며 fragment 단위 TF32 변환과 단순한 tile scheduling으로 인해 큰 matrix에서 충분한 성능 확장이 이루어지지 않았기 때문으로 해석된다.

이 실험을 통해 CUDA Core 기반 FP32 Jacobi iteration은 CPU 대비 큰 성능 향상을 보이면서도 CPU와 유사한 precision을 유지함을 확인했다. Tensor Core 기반 TF32 Jacobi iteration은 매우 빠른 연산이 가능하지만 엄격한 residual 기준에서는 precision 한계가 나타났다. 따라서 Tensor Core는 높은 처리량이 필요한 경우 유리하지만 매우 작은 residual을 요구하는 문제에서는 CUDA Core FP32 방식이 더 안정적이다.

이번 과제를 통해 Jacobi iteration이 matrix multiply-accumulate 형태인 $D = A*B + C$ 로 표현될 수 있고 GPU에서 병렬화하면 CPU 대비 큰 성능 향상을 얻을 수 있음을 확인했다. 특히 CUDA Core를 사용한 FP32 tiled GEMM 구현은 shared memory를 활용하여 global memory 접근을 줄이고 CPU보다 훨씬 빠른 시간 안에 CPU와 유사한 수준의 residual을 얻을 수 있다는 것을 확인할 수 있었다.

이번 실험에서 가장 인상적이었던 점은 performance와 precision 사이의 trade-off였다. Tensor Core는 작은 matrix size에서 매우 빠른 성능을 보였지만 TF32 precision의 한계로 인해 $1e-7$ 수준의 엄격한 residual 조건은 만족하지 못하였다. 반면 CUDA Core FP32 방식은 Tensor Core보다 느릴 수 있지만 CPU와 거의 같은 residual 수준까지 수렴할 수 있다는 것을 확인했다.

```
===== HOST =====
>>> Trying to solve for X in UX = V of size 256 using 20 maximum Jacobi iterations...
>>> The final relative residual error after 8 iterations ||R||/||V|| = ||UX - V||/||V|| is 3.052558e-08.
>>> HOST elapsed time: 787.862 ms
>>> HOST residual recomputed on CPU: 3.052558e-08

===== CUDA CORE =====
>>> Trying to solve for X in UX = V of size 256 using 20 maximum Jacobi iterations on CUDA Cores...
>>> residual error after 8 iterations ||R||/||V|| = ||UX - V||/||V|| is 3.649433e-08.
>>> CUDA Core elapsed time: 112.909 ms
>>> CUDA Core residual recomputed on CPU: 3.649433e-08

===== TENSOR CORE =====
>>> Trying to solve for X in UX = V of size 256 using 20 maximum Jacobi iterations on Tensor Cores...
>>> residual error after 20 iterations ||R||/||V|| = ||UX - V||/||V|| is 1.793526e-05.
>>> Tensor Core elapsed time: 5.608 ms
>>> Tensor Core residual recomputed on CPU: 1.793526e-05

===== SUMMARY =====
HOST      : 787.862 ms
CUDA Core : 112.909 ms
Tensor Core : 5.608 ms
>>> HOST Jacobi iteration completed successfully.
>>> CUDA Core Jacobi iteration completed successfully.
>>> Tensor Core Jacobi iteration failed to converge (Error Code: 1).

===== HOST =====
>>> Trying to solve for X in UX = V of size 512 using 20 maximum Jacobi iterations...
>>> The final relative residual error after 8 iterations ||R||/||V|| = ||UX - V||/||V|| is 2.977081e-08.
>>> HOST elapsed time: 6939.589 ms
>>> HOST residual recomputed on CPU: 2.977081e-08

===== CUDA CORE =====
>>> Trying to solve for X in UX = V of size 512 using 20 maximum Jacobi iterations on CUDA Cores...
>>> residual error after 8 iterations ||R||/||V|| = ||UX - V||/||V|| is 3.490851e-08.
>>> CUDA Core elapsed time: 136.682 ms
>>> CUDA Core residual recomputed on CPU: 3.490851e-08

===== TENSOR CORE =====
>>> Trying to solve for X in UX = V of size 512 using 20 maximum Jacobi iterations on Tensor Cores...
>>> residual error after 20 iterations ||R||/||V|| = ||UX - V||/||V|| is 1.264876e-05.
>>> Tensor Core elapsed time: 22.017 ms
>>> Tensor Core residual recomputed on CPU: 1.264876e-05

===== SUMMARY =====
HOST      : 6939.589 ms
CUDA Core : 136.682 ms
Tensor Core : 22.017 ms
>>> HOST Jacobi iteration completed successfully.
>>> CUDA Core Jacobi iteration completed successfully.
>>> Tensor Core Jacobi iteration failed to converge (Error Code: 1).
```

```

===== HOST =====
>>> Trying to solve for X in UX = V of size 1024 using 20 maximum Jacobi iterations...
.....
>>> The final relative residual error after 6 iterations  $\|R\|/\|V\| = \|UX - V\|/\|V\|$  is 4.960132e-08.
>>> HOST elapsed time: 54555.988 ms
>>> HOST residual recomputed on CPU: 4.960132e-08

===== CUDA CORE =====
>>> Trying to solve for X in UX = V of size 1024 using 20 maximum Jacobi iterations on CUDA Cores...
.....
>>> residual error after 6 iterations  $\|R\|/\|V\| = \|UX - V\|/\|V\|$  is 5.234583e-08.
>>> CUDA Core elapsed time: 189.479 ms
>>> CUDA Core residual recomputed on CPU: 5.234583e-08

===== TENSOR CORE =====
>>> Trying to solve for X in UX = V of size 1024 using 20 maximum Jacobi iterations on Tensor Cores...
.....
>>> residual error after 20 iterations  $\|R\|/\|V\| = \|UX - V\|/\|V\|$  is 1.008106e-05.
>>> Tensor Core elapsed time: 375.566 ms
>>> Tensor Core residual recomputed on CPU: 1.008106e-05

===== SUMMARY =====
HOST          : 54555.988 ms
CUDA Core     : 189.479 ms
Tensor Core   : 375.566 ms
>>> HOST Jacobi iteration completed successfully.
>>> CUDA Core Jacobi iteration completed successfully.
>>> Tensor Core Jacobi iteration failed to converge (Error Code: 1).

===== HOST =====
>>> Trying to solve for X in UX = V of size 2048 using 20 maximum Jacobi iterations...
.....
>>> The final relative residual error after 6 iterations  $\|R\|/\|V\| = \|UX - V\|/\|V\|$  is 3.037843e-08.
>>> HOST elapsed time: 521829.156 ms
>>> HOST residual recomputed on CPU: 3.037843e-08

===== CUDA CORE =====
>>> Trying to solve for X in UX = V of size 2048 using 20 maximum Jacobi iterations on CUDA Cores...
.....
>>> residual error after 6 iterations  $\|R\|/\|V\| = \|UX - V\|/\|V\|$  is 3.589873e-08.
>>> CUDA Core elapsed time: 557.857 ms
>>> CUDA Core residual recomputed on CPU: 3.589873e-08

===== TENSOR CORE =====
>>> Trying to solve for X in UX = V of size 2048 using 20 maximum Jacobi iterations on Tensor Cores...
.....
>>> residual error after 20 iterations  $\|R\|/\|V\| = \|UX - V\|/\|V\|$  is 1.214108e-05.
>>> Tensor Core elapsed time: 1239.549 ms
>>> Tensor Core residual recomputed on CPU: 1.214108e-05

===== SUMMARY =====
HOST          : 521829.156 ms
CUDA Core     : 557.857 ms
Tensor Core   : 1239.549 ms
>>> HOST Jacobi iteration completed successfully.
>>> CUDA Core Jacobi iteration completed successfully.
>>> Tensor Core Jacobi iteration failed to converge (Error Code: 1).

```